Rapport de TP

Sommaire

1. Partie Simulation **2**
1.1 Prise en main et clignotement d'une LED 2
1.2. Mise en œuvre d'un bargraphe 3
1.3. Intégration du capteur DHT22 3
1.4. Intégration de l'afficheur LCD I2C 4
1.5. Réalisation du capteurTempHumiX 4
2. Communication Wi-Fi avec le protocole MQTT et analyse réseau **5**
2.1. Connexion au broker et échanges de messages 5
2.2. Expérimentation des fonctionnalités MQTT 5
2.3. Adaptation du programme et affichage local Gestion du RTC et connexion Wi-Fi 6
2.4. Publication et réception des commandes MQTT 6
2.5. Analyse du trafic réseau avec Wireshark 7
Réponses au question de la partie: Analyse du réseau IoT communicant en MOTT 8
Réponse aux questions générales sur l'architecture IoT **9**
3. Construction IHM **10**
3.1 Conception de l'interface graphique 11
3.2 Consultation de l'historique 12
3.4 Conception de la boîte de dialogue 13
3.5 Traitement et génération des courbes 14
Difficultés rencontrées et solutions **15**

1. Partie Simulation

Ce projet consiste à concevoir et simuler, à l'aide du logiciel SimulIDE, un boîtier intelligent de mesure environnementale nommé TempHumiX. Basé sur une carte Arduino Uno, ce dispositif permet de mesurer la température et l'humidité, d'afficher les résultats localement et de transmettre les données via une liaison série.

La réalisation s'est déroulée progressivement, de la découverte des composants électroniques jusqu'à l'intégration d'un système complet associant acquisition, affichage et communication.

Deux principaux outils ont été utilisés :

- **SimulIDE:** logiciel de simulation électronique permettant de concevoir et de tester les circuits autour du microcontrôleur Arduino Uno, ainsi que de visualiser les échanges sur le port série.
- **Arduino IDE:** est un logiciel de développement (environnement de programmation) permettant de programmer des cartes électroniques comme l'Arduino Uno
- **Git Bash et GitHub :** utilisés pour assurer le suivi des versions du projet, organiser les fichiers et conserver l'historique des modifications du code.

1.1 Prise en main et clignotement d'une LED

La première manipulation a consisté à réaliser un circuit simple associant une carte Arduino Uno, une LED et une résistance de protection de 100 Ω. La LED est connectée à la broche numérique 2.

Le programme clignotement_led.ino commande alternativement l'allumage et l'extinction de la LED avec un intervalle de 500 ms grâce à la fonction delay(500).

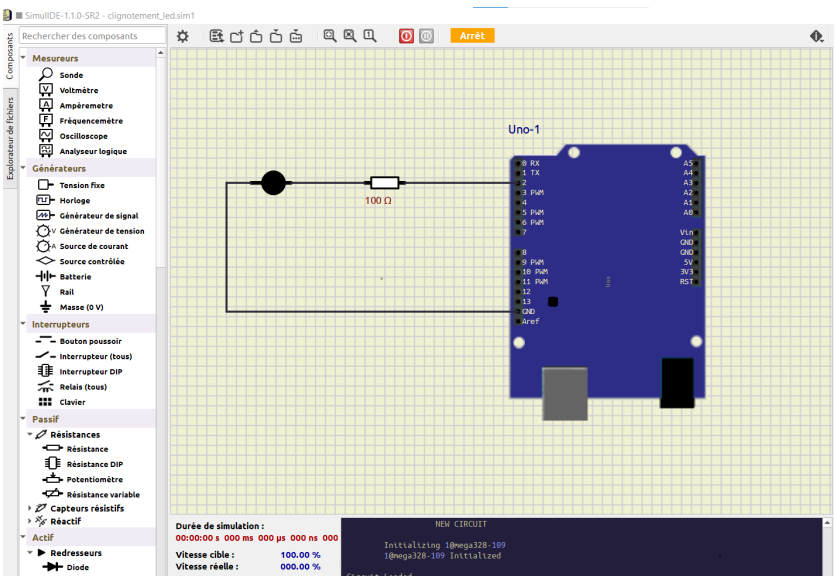


Figure 1 : Simulation du clignotement d'une LED sous SimulIDE.

1.2. Mise en œuvre d'un bargraphe

La deuxième étape a porté sur l'intégration d'un bargraphe composé de 10 LED, associé à un réseau de résistances DIP de 100 Ω et relié aux sorties numériques de l'Arduino.

Le programme réalise un chenillard lumineux en activant successivement les LED avec un intervalle de 100 ms.

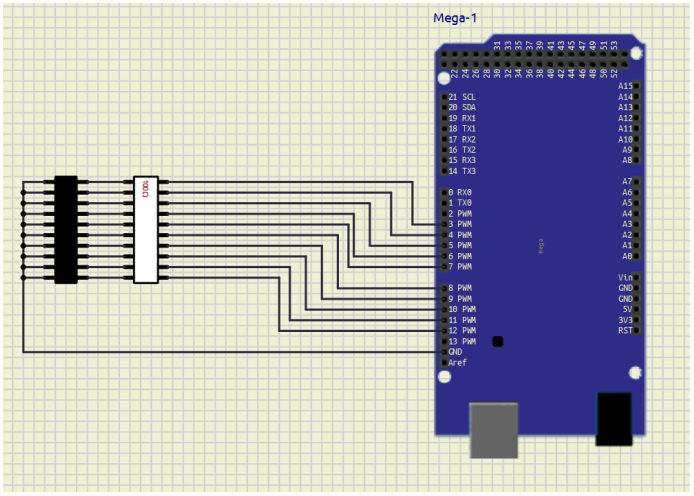


Figure 2 : Réalisation et simulation du bargraphe sous SimulIDE.

1.3. Intégration du capteur DHT22

Un capteur DHT22 a ensuite été ajouté afin de mesurer la température et l'humidité ambiantes. Deux LED de signalisation, verte et rouge, permettent d'indiquer le niveau de température détecté.

Le fonctionnement repose sur un seuil de 20 °C : la LED verte s'allume lorsque la température est inférieure à ce seuil, tandis que la LED rouge s'active lorsqu'elle le dépasse.

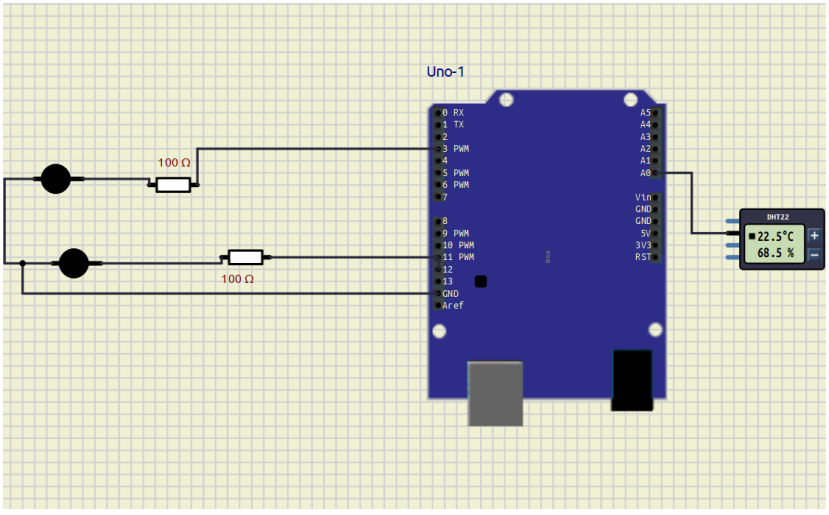


Figure 3 : Simulation du capteur DHT22 et du système de signalisation.

1.4. Intégration de l'afficheur LCD I2C

Un afficheur LCD 16x2 utilisant le contrôleur AIP31068 a été intégré au circuit. Sa connexion à l'Arduino s'effectue via le bus I2C.

L'affichage a été testé à l'aide de la bibliothèque LiquidCrystal_AIP31068_I2C, permettant de présenter des messages textuels ainsi que les valeurs mesurées par le capteur.

Exemple d'affichage : température de 22,5 °C et humidité de 68,5 %.

1.5. Réalisation du capteurTempHumiX

Après la validation individuelle des différents composants, leur intégration a permis de constituer le système complet TempHumiX autour d'une carte Arduino Uno.

Le montage final comprend :

- Un capteur DHT22 pour mesurer la température et l'humidité.
- Un afficheur LCD I2C 16x2 pour visualiser les mesures.
- Un bargraphe de 10 LED pour la représentation visuelle.
- Des LED indicatrices pour les états de fonctionnement (Power, vie, Rx et Tx).
- Une liaison série RS232 émulée pour transmettre les données vers un équipement externe.

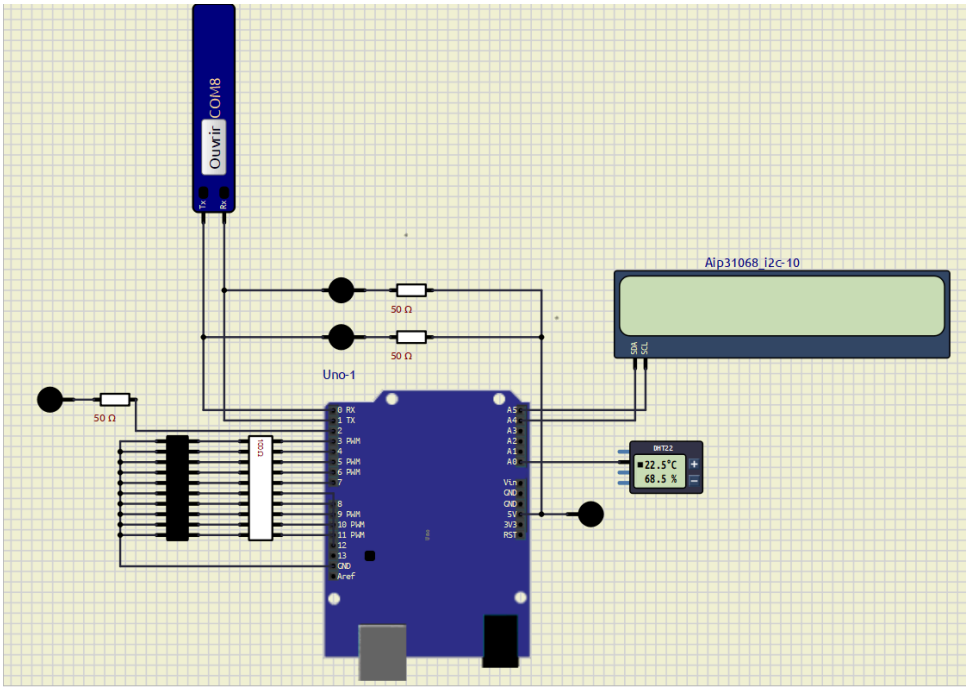


Figure 4 : Capteur TempHumiX.

Ce travail a permis de concevoir et de simuler un système embarqué complet dédié à la mesure environnementale. L'intégration du capteur DHT22, de l'afficheur LCD, du bargraphe et des indicateurs lumineux a abouti à la réalisation du boîtier intelligent TempHumiX.

Cette première étape a également permis de valider l'acquisition, l'affichage et la transmission série des données. Elle constitue ainsi une base pour le déploiement du dispositif physique et son évolution vers une architecture IoT intégrant une communication Wi-Fi et le protocole MQTT.

2. Communication Wi-Fi avec le protocole MQTT et analyse réseau

Ce TP a pour objectif de mettre en place une communication entre le capteur intelligent TempHumiX, basé sur une carte Arduino UNO R4 WiFi, et un ordinateur distant à l'aide du protocole MQTT.

Les travaux réalisés portent sur la prise en main de MQTT avec MQTTX et Mosquitto, l'adaptation du programme Arduino, la gestion du Wi-Fi et de la synchronisation horaire NTP, ainsi que l'analyse des échanges réseau avec Wireshark.

2.1. Connexion au broker et échanges de messages

La première étape consiste à établir une connexion au broker Eclipse Mosquitto à l'aide du client MQTTX.

Les paramètres utilisés sont :

- Adresse IP : 192.168.1.1
- Port : 1883
- Identifiant : mos_user
- Mot de passe : mos_user

Un topic de test personnalisé (TESTS/42904) est créé afin de vérifier la publication et la réception des messages. (Voir figure 6)

2.2. Expérimentation des fonctionnalités MQTT

Plusieurs tests ont permis de vérifier le fonctionnement des abonnements et des publications :

- **Souscription à un topic d'un autre poste :** les messages sont reçus si les règles ACL du broker autorisent cet accès.
- **Souscription à TESTS/# :** le caractère générique # permet de recevoir les messages de tous les sous-topics de la branche TESTS/.
- **Publication avec Retain :** le broker conserve le dernier message publié et le transmet immédiatement aux nouveaux abonnés. La publication d'un message vide avec Retain permet de supprimer cette valeur conservée.

2.3. Adaptation du programme et affichage local Gestion du RTC et connexion Wi-Fi

Le programme initial a été adapté pour fonctionner sur la carte Arduino UNO R4 WiFi. L'afficheur LCD RGB Backlight a été intégré afin de présenter localement les informations du capteur. L'ancien système d'horodatage logiciel a été remplacé par l'utilisation de l'horloge temps réel (RTC) intégrée à la carte. La bibliothèque WiFiS3 permet de connecter le capteur au réseau Wi-Fi « IoT IMT Nord Europe ».

Le RTC assure l'affichage de la date et de l'heure sur l'écran LCD. Une synchronisation automatique est également réalisée avec le serveur NTP de l'établissement à l'aide de la bibliothèque NTPClient.

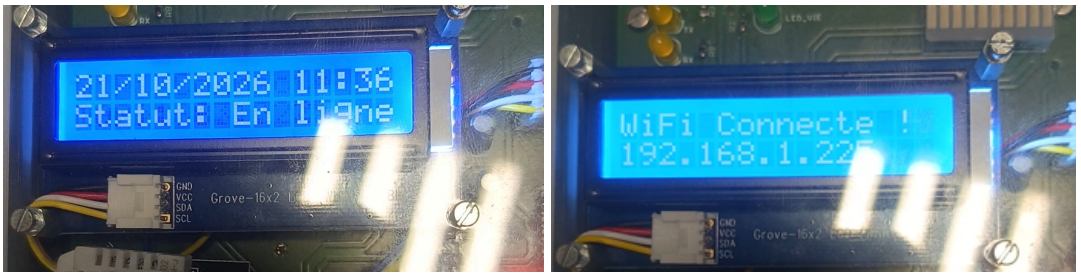


Figure 5 : Affichage des informations du capteur après connexion Wi-Fi et synchronisation NTP.

2.4. Publication et réception des commandes MQTT

Le capteur publie ses informations sur quatre topics : VERSION, TEMPERATURE, HUMIDITY et GET_DATE, associés à l'identifiant du poste.

Les publications sont effectuées périodiquement, avec une optimisation permettant de limiter les transmissions lorsque les mesures ne changent pas.

Le capteur souscrit également aux topics de commande afin de permettre son pilotage à distance :

- Synchronisation de la date et de l'heure.
- Démarrage et arrêt des mesures.
- Commande du bargraphe à 10 LED.
- Modification de la période de clignotement de la LED.

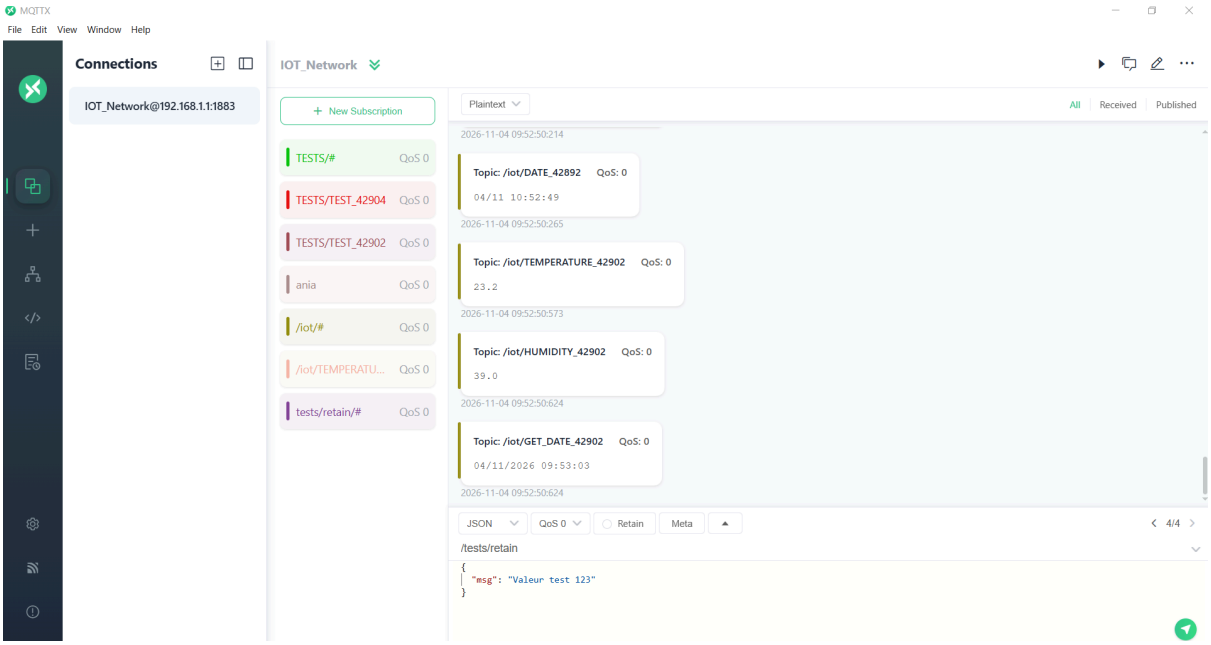


Figure 6 : Échanges MQTT entre le capteur TempHumiX et le broker.

2.5. Analyse du trafic réseau avec Wireshark

L'analyse du trafic a été réalisée avec Wireshark en utilisant le filtre tcp port 1883, puis le filtre d'affichage mqtt.

Les principales trames observées sont :

- **CONNECT / CONNACK :** établissement et confirmation de la connexion au broker.
- **SUBSCRIBE / SUBACK :** demande et confirmation d'un abonnement.
- **PUBLISH :** transmission des données et des commandes.
- **PINGREQ / PINGRESP :** maintien de la connexion grâce au mécanisme Keep Alive.
- **DISCONNECT :** fermeture de la session MQTT.

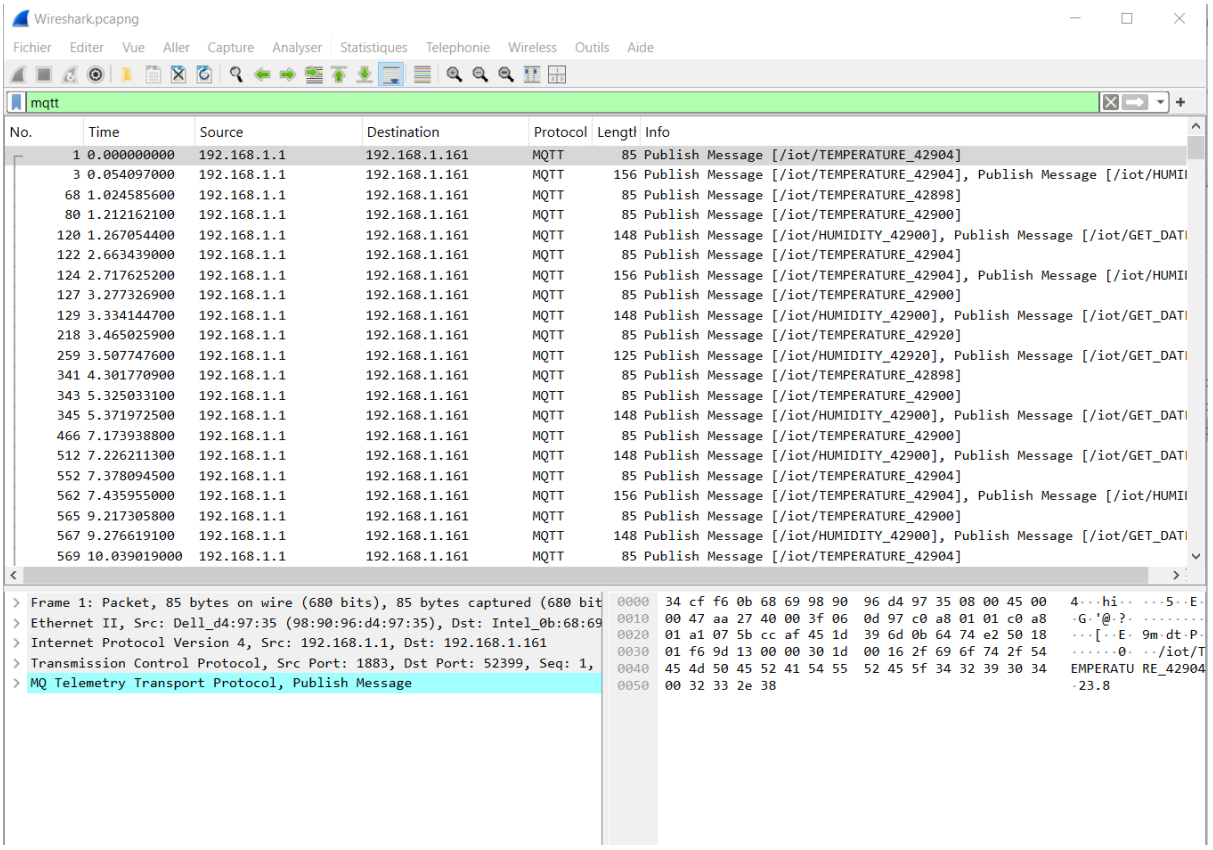


Figure 7 : Analyse des échanges MQTT sur le port 1883 avec Wireshark.

Réponses au question de la partie: Analyse du réseau IoT communicant en MOTT

Où apparaît le mot de passe mos_user dans la capture Wireshark ? Que peut-on en déduire concernant la sécurité des communications sur le port 1883 ?

Le mot de passe apparaît en clair dans le paquet CONNECT. En effet, le port 1883 ne chiffre pas les échanges MQTT. Les identifiants peuvent donc être interceptés lors de leur transmission. Pour sécuriser les communications, il est préférable d'utiliser MQTTS avec TLS, généralement sur le port 8883.

Quelle est la taille d'un paquet PUBLISH contenant une mesure de température ? Comment se compare-t-elle à celle d'une requête HTTP équivalente ?

Un paquet PUBLISH MQTT est très léger (environ 85 octets) comme on peut le voir dans figure 7 . À l'inverse, une requête HTTP équivalente est beaucoup plus lourde. MQTT est donc bien plus adapté aux réseaux IoT à faible bande passante.

Que représente le paramètre Keep Alive dans MQTT et que se passe-t-il lorsqu'aucun message n'est échangé pendant l'intervalle défini ?

Le Keep Alive définit la durée maximale entre deux échanges du client et du broker. En l'absence de communication, le client envoie un paquet PINGREQ auquel le broker répond par PINGRESP. Sans réponse, la connexion est considérée comme interrompue.

Réponse aux questions générales sur l'architecture IoT

1 . En considérant l'ensemble de l'architecture IoT mise en place, notamment le capteur, le broker MQTT et le routeur Teltonika RUTX50, comment les informations sont-elles acheminées selon les modèles OSI et TCP/IP ?

Les données sont transmises depuis le capteur à travers plusieurs couches :

- Application : MQTT assure les échanges de messages.
- Transport : TCP garantit la fiabilité des transmissions.
- Réseau : IP assure l'adressage et le routage des paquets.
- Liaison et physique : le Wi-Fi permet la transmission sans fil.

Le routeur Teltonika RUTX50 assure l'interconnexion entre les équipements et le broker Mosquitto, qui redistribue ensuite les messages aux clients abonnés.

2 . Quels sont les principaux services réseau proposés par le routeur Teltonika RUTX50 ? À quel niveau de l'architecture IoT peut-on le situer et quel est son rôle dans l'interconnexion des équipements ?

Le RUTX50 propose des services de routage, de pare-feu, de DHCP et de VPN. Il se situe à la périphérie du réseau, au niveau Edge/Fog Computing. Il joue le rôle de passerelle entre le réseau local des capteurs et les réseaux externes, notamment Wi-Fi, Ethernet et 4G/5G.

3 . Le routeur Teltonika RUTX50 est-il équipé d'un microcontrôleur (MCU) ou d'un microprocesseur (MPU) ? Quel système d'exploitation utilise-t-il ?

Le RUTX50 est équipé d'un microprocesseur (MPU), capable de faire fonctionner un système d'exploitation complet. Il utilise RutOS, un système basé sur Linux et OpenWrt.

4 . Les connexions Wi-Fi sur le routeur RUTX50 sont-elles illimitées ? Quels sont les risques liés à un nombre important de connexions et quelle topologie réseau est utilisée ?

Non, le nombre de connexions est limité par les capacités matérielles du routeur et la bande passante disponible. Une surcharge peut entraîner des ralentissements ou des attaques par déni de service (DoS). D'autres risques existent, comme les intrusions et l'interception des communications. La topologie utilisée est généralement en étoile, avec le routeur comme point d'accès central.

5 . Quelles sont les principales mesures de sécurité intégrées dans les versions récentes du firmware du routeur ?

Les principales mesures comprennent le pare-feu, l'utilisation de mots de passe robustes, la désactivation des services obsolètes, le chiffrement des connexions Wi-Fi, notamment avec WPA3 selon la configuration matérielle, ainsi que les mises à jour régulières du firmware pour corriger les vulnérabilités.

6 . Que révèlent les logs du broker MQTT Mosquitto concernant son démarrage et les éventuels problèmes de configuration ?

Les logs indiquent un démarrage réussi de Mosquitto (version 2.1.2), avec une ouverture des connexions IPv4 et IPv6 sur le port 1883.

Cependant, plusieurs avertissements signalent des permissions trop larges sur les fichiers de mots de passe et d'ACL, ainsi qu'une configuration ACL ne faisant pas appel aux variables dynamiques.

Ces avertissements montrent qu'un renforcement des permissions et des règles de contrôle d'accès est nécessaire.

3. Construction IHM

Ce projet porte sur la réalisation d'une architecture complète d'Internet des Objets (IdO). Il consiste à acquérir des données environnementales à partir d'un capteur intelligent (température, humidité, version et horodatage), à les transmettre via MQTT, à les enregistrer dans une base de données SQLite et à les visualiser à travers une Interface Homme-Machine (IHM) développée en Python avec PySide6.

Le protocole MQTT (*Message Queuing Telemetry Transport*) assure les échanges de données entre le capteur et la station de travail. Le fichier client_mqtt.py regroupe cette communication au sein de la classe ClientMQTT.

La classe ClientMQTT repose sur la bibliothèque Paho-MQTT et utilise les principales méthodes suivantes :

- **connect_mqtt() :** établit la connexion au broker à partir de son adresse, de son port et des identifiants d'authentification.
- **subscribe_mqtt() :** souscrit aux topics du capteur afin de recevoir les informations relatives à sa version, sa température, son humidité et son horodatage. L'identifiant du capteur est extrait automatiquement du topic reçu.
- **publish_mqtt() :** permet de transmettre des commandes au capteur, notamment pour le pilotage du bargraphe et les différentes actions à distance.

La réception continue des messages MQTT nécessite une écoute permanente. Une exécution directe dans le thread principal risquerait de bloquer l'interface graphique. Pour éviter ce blocage, la classe ThreadMQTT, héritant de Thread, assure la réception des messages en arrière-plan.

Elle exécute périodiquement self.mqtt.wait_mqtt(timeout=1.0), permettant de traiter les messages sans interrompre l'affichage de l'IHM.

Ce thread gère également le cycle de vie de la connexion SQLite : ouverture au démarrage avec connect_sqlite() et fermeture à l'arrêt avec disconnect_sqlite().

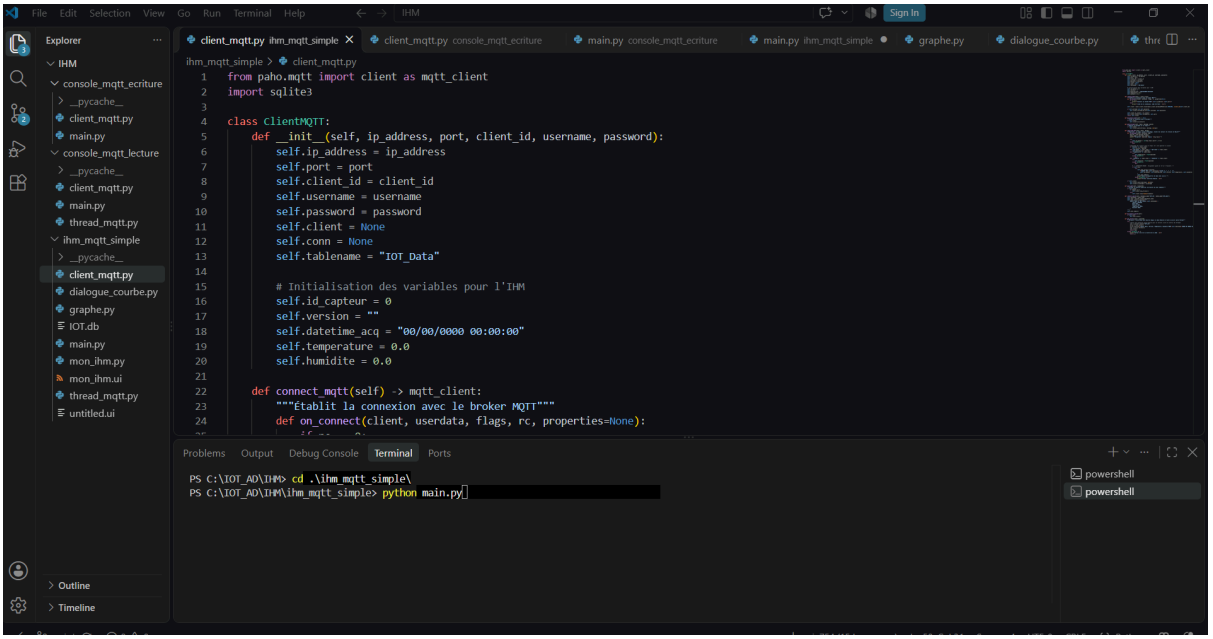


Figure 8: Réalisation sur vs code

3.1 Conception de l'interface graphique

L'interface a été conçue avec PySide6 Designer, puis convertie en code Python. Elle est organisée autour de quatre fonctionnalités :

- **Communication MQTT :** saisie des paramètres du broker et gestion de la connexion.
- **Affichage des mesures :** visualisation de la température, de l'humidité et de la date d'acquisition.
- **Commandes du capteur :** synchronisation NTP, démarrage et arrêt des mesures, réglage des modes de clignotement de la LED et pilotage du bargraphe.
- **Historique :** affichage des mesures précédemment enregistrées dans un tableau QTableWidget.

Le fichier main.py contient la classe MonIHM, qui assure la liaison entre les composants graphiques et les fonctionnalités du système.

Un QTimer, configuré à 1 000 ms, déclenche la méthode display() afin d'actualiser les données affichées.

Les boutons sont associés aux méthodes de commande correspondantes grâce aux signaux Qt (clicked.connect et returnPressed.connect). Les commandes sont ensuite publiées sur les topics MQTT appropriés.

Par exemple, les commandes numériques 10, 20 et 30 permettent respectivement de gérer la synchronisation NTP et le contrôle des mesures, tandis que le bargraphe reçoit sa valeur sur le topic /iot/BARGRAPH_<id_pc>.

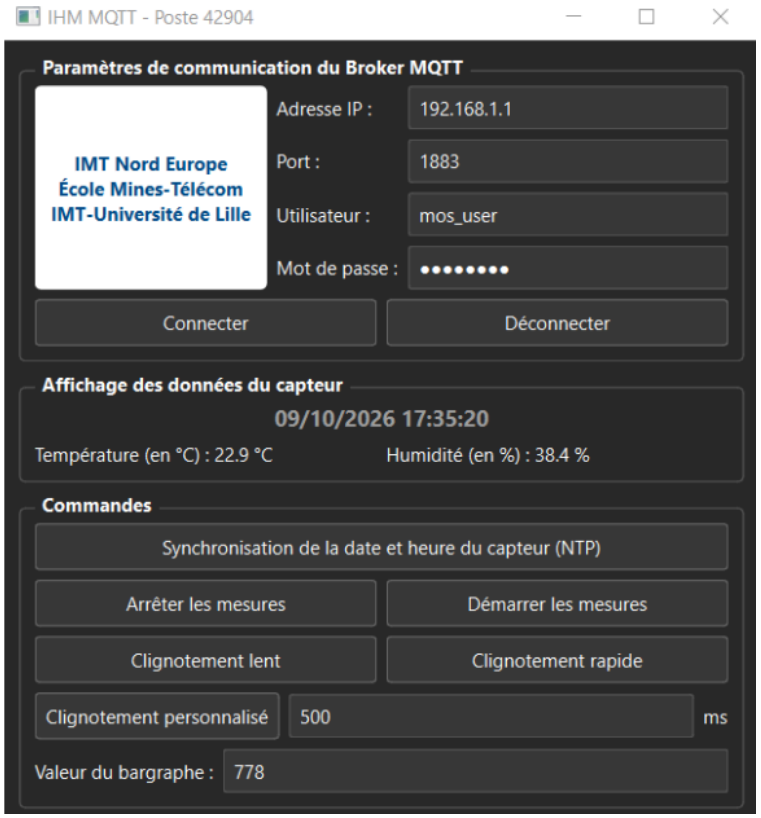


Figure 9: Capture d’écran de l'interface graphique

Les mesures reçues sont automatiquement enregistrées dans une base de données locale nommée IOT.db. Chaque enregistrement associe ainsi un capteur à ses mesures et à leur horodatage.

3.2 Consultation de l'historique

La méthode get_history(limit=50) permet de récupérer les dernières mesures enregistrées.

Les résultats sont ensuite affichés dans le tableau de l'IHM à l'aide de la fonction chargerHistorique().

Une connexion SQLite indépendante et temporaire est utilisée pour cette consultation afin de limiter les conflits d'accès entre les différents threads.

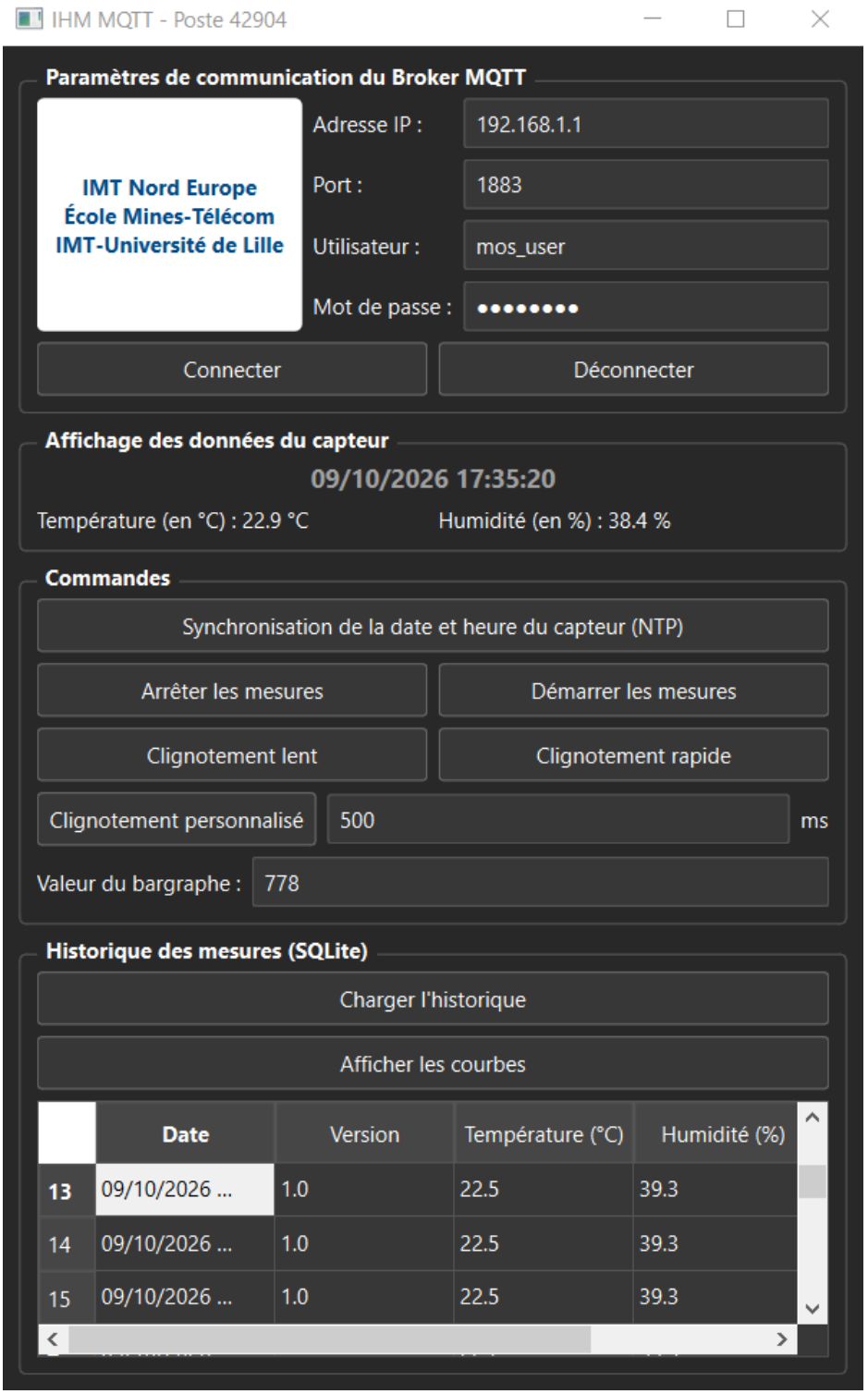


Figure 9: Capture d’écran de l'interface graphique avec l’historique des mesures

3.4 Conception de la boîte de dialogue

Une seconde interface graphique a été réalisée avec PySide6 Designer, dans le fichier graphe.ui, afin de permettre la visualisation des mesures enregistrées sur une période donnée.

Cette boîte de dialogue intègre :

- Deux sélecteurs QDateTimeEdit pour définir les dates et heures de début et de fin.
- Un composant QGraphicsView destiné à afficher les courbes.
- Un bouton btnLoadData pour charger les données et un bouton btnClearData pour effacer le graphique.

3.5 Traitement et génération des courbes

La classe CourbeDialog assure la récupération des données et leur représentation graphique.

Le traitement repose sur trois étapes :

- **Récupération des données :** une requête SQL interroge la base IOT.db afin d'extraire les dates, températures et humidités comprises dans l'intervalle sélectionné (BETWEEN ... AND ...).
- **Génération des courbes :** la bibliothèque Matplotlib est utilisée pour représenter l'évolution des mesures. La température est affichée en rouge avec des marqueurs circulaires (o), tandis que l'humidité apparaît en bleu avec des marqueurs en forme de croix (+).

Cette fonctionnalité permet ainsi de consulter l'évolution historique des données environnementales sur une période choisie.

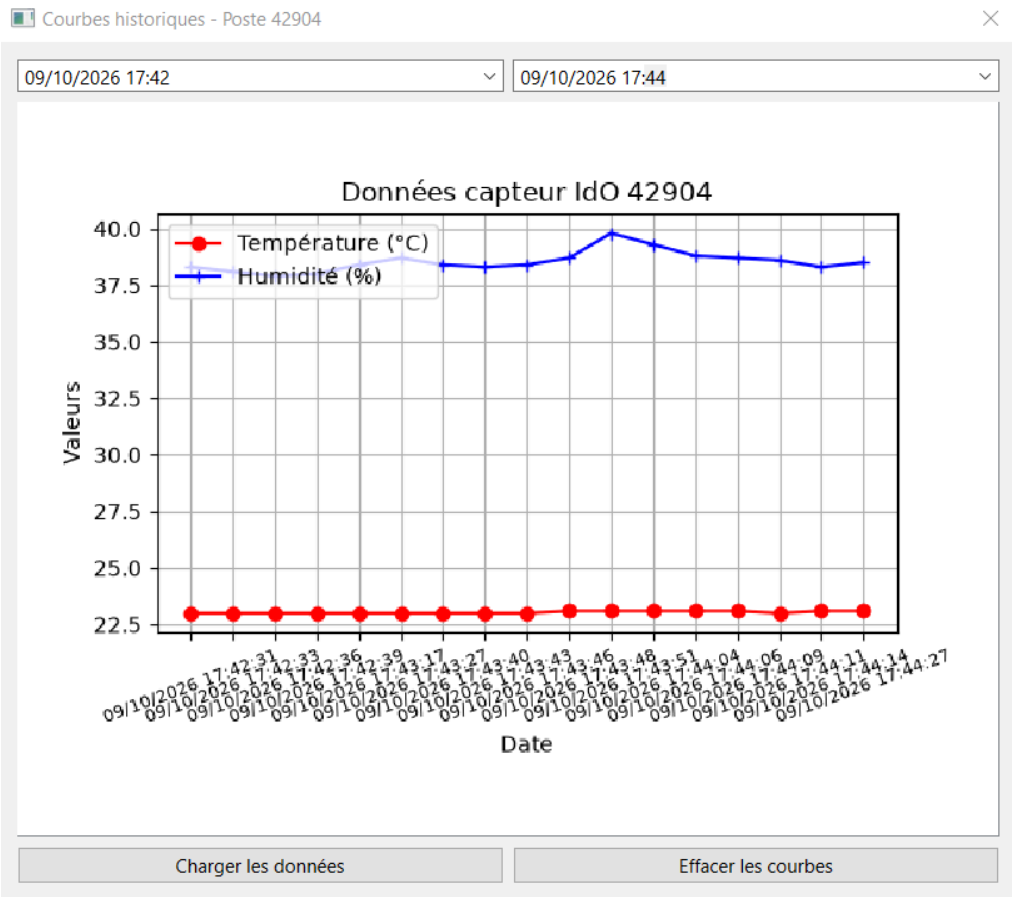


Figure 10 : Capture d’écran du graphique avec les courbes historiques de l’humidité et de la température

Ce projet a permis de mettre en œuvre une chaîne complète de supervision IoT, depuis l'acquisition des données jusqu'à leur visualisation et leur archivage.

L'association de Python, MQTT, des threads, de SQLite et de PySide6 a permis de développer une application de supervision fonctionnelle, capable de recevoir des mesures, de conserver leur historique et de piloter le capteur à distance.

Difficultés rencontrées et solutions

Compatibilité de la bibliothèque LCD : L'utilisation de la bibliothèque LiquidCrystal_AIP31068_I2C a posé des difficultés de compatibilité avec l'environnement de développement. Il a donc été nécessaire de rechercher et d'utiliser une bibliothèque alternative compatible avec l'afficheur.

Chargement du programme dans SimulIDE : Le programme Arduino ne pouvait pas être directement exécuté dans SimulIDE. Il a fallu passer par l'IDE Arduino pour compiler le code et générer un fichier .hex, puis charger ce fichier dans le composant Arduino du simulateur afin de lancer la simulation.

Correspondance des boutons de l'IHM : Lors de l'intégration de l'interface graphique développée avec PySide6 Designer, certains boutons ne fonctionnaient pas correctement en raison de différences entre leurs noms dans le fichier .ui et ceux utilisés dans le code Python. Une vérification des identifiants des widgets et de leurs connexions aux fonctions a permis de corriger ces erreurs.